Healthcare Encounter Reporting & Data Quality

Detailed Project Report

Synthea synthetic patient records · PostgreSQL 16 · Python

1,141	64,622	9	40	137 / 150
synthetic patients	encounters	related tables	validation checks	injected errors detected

Prepared by Suman Oraon · October 2026

Synthetic data. All patients and encounters were generated with Synthea, the open-source synthetic patient generator from The MITRE Corporation (seed 20261012, Philadelphia, Pennsylvania). No real patient data is used. Facility and payer names come from Synthea's public reference files; their encounter volumes are simulated.

Contents

1. Executive summary
2. Data source and approach
3. Table relationships
4. SQL reporting views
5. Encounter volume and visit types
6. Join reconciliation
7. Validation checks
8. Documented exceptions
9. Testing the checks against injected errors
10. Limitations and next steps
11. Reproducing the analysis

1. Executive summary

This project builds a PostgreSQL reporting layer and a data-quality layer on top of Synthea synthetic health records: 1,141 patients, 64,622 encounters and 280,280 clinical events across 9 related tables. It answers two questions. What does encounter volume look like by class and visit type? And can the checks be trusted to catch the errors that break reporting?

Key results

- **8 SQL reporting views** cover encounter detail, monthly volume, class and visit-type summaries, age and gender, payer, patient utilization and organization volume.
- **Encounter counts reconcile in the production data.** All 64,622 encounters survive every join, and every roll-up view adds back to the same total.
- **The reconciliation catches fan-out.** In the test dataset, duplicate patient and provider IDs would inflate the reporting view by 2,500 rows (3.9%) if counts were not checked before and after each join.
- **40 validation checks** in four groups: missing keys (11), orphan records (10), duplicate identifiers (8) and inconsistent dates (11).
- **137 of 150 deliberately introduced errors were detected** in a separate test dataset: all 137 structural errors, each by the check designed for it, with no false positives. The 13 misses are plausible-but-wrong values included on purpose to show what structural checks cannot catch.
- **4 genuine source exceptions** (305 rows) were found in the untouched Synthea output and documented, and two check rules were adjusted with a written rationale.

Encounter highlights (Oct 2016 – Sep 2026)

- Ambulatory visits are 53.8% of encounters and wellness visits 19.6%. Inpatient stays are 1.2% of volume but average 19,967 dollars per claim.
- Annual volume holds at about 3,900–4,700 encounters, except 2021 (5,341), when COVID-19 vaccination added more than 1,100 visits.
- Government payers (Medicaid, Medicare, dual eligible) cover 61% of encounters; 8% are uninsured.

2. Data source and approach

Synthea simulates the medical history of a synthetic population from birth to the reference date, using clinical care maps for common conditions. It exports standard tables in the same shape as an EHR extract.

Setting	Value
Generator	Synthea (official release jar, build d9d07a6)
Command	-p 1000 -s 20261012 -cs 20261012 -r 20260930, Pennsylvania / Philadelphia
Population	1,141 patients (1,000 alive at the reference date plus 141 who died during the simulation)
History	Encounters from 1917-01-31 to 2026-10-16; reporting window Oct 2016 – Sep 2026 (44,424 encounters)
Export	CSV, 9 of 18 tables used; identifier-style fields (SSN, license, passport, names, street address) removed

Table 1. Generation settings (synthea/generate_synthea.sh)

Pipeline

Step	Script	What it does
1	synthea/generate_synthea.sh	Downloads Synthea and generates the seeded population
2	python/01_prepare_raw.py	Selects 9 tables, drops identifier fields, adds a stable _row_id, writes data/raw/*.csv.gz
3	python/02_load_postgres.py	Loads schema synthea, builds 8 reporting views and the join reconciliation
4	python/03_run_dq_checks.py	Runs the 40 checks; every failing row goes to dq_exceptions
5	python/04_build_test_dataset.py	Builds a separate test copy with 150 logged errors
6	02 + 03 on schema synthea_test	Loads the test copy and runs the same views, reconciliation and checks
7	python/05_score_detection.py	Scores detection per error, traces every secondary flag
8	python/06–07	Exports reporting outputs and builds this report

Table 2. Pipeline (run_all.sh runs every step)

Design choice: keys are checked, not enforced. The tables are loaded without PRIMARY KEY or FOREIGN KEY constraints so that the data loads as received. A constraint would reject a bad row at load time and hide the problem; the checks instead find, count and trace every failing row.

3. Table relationships

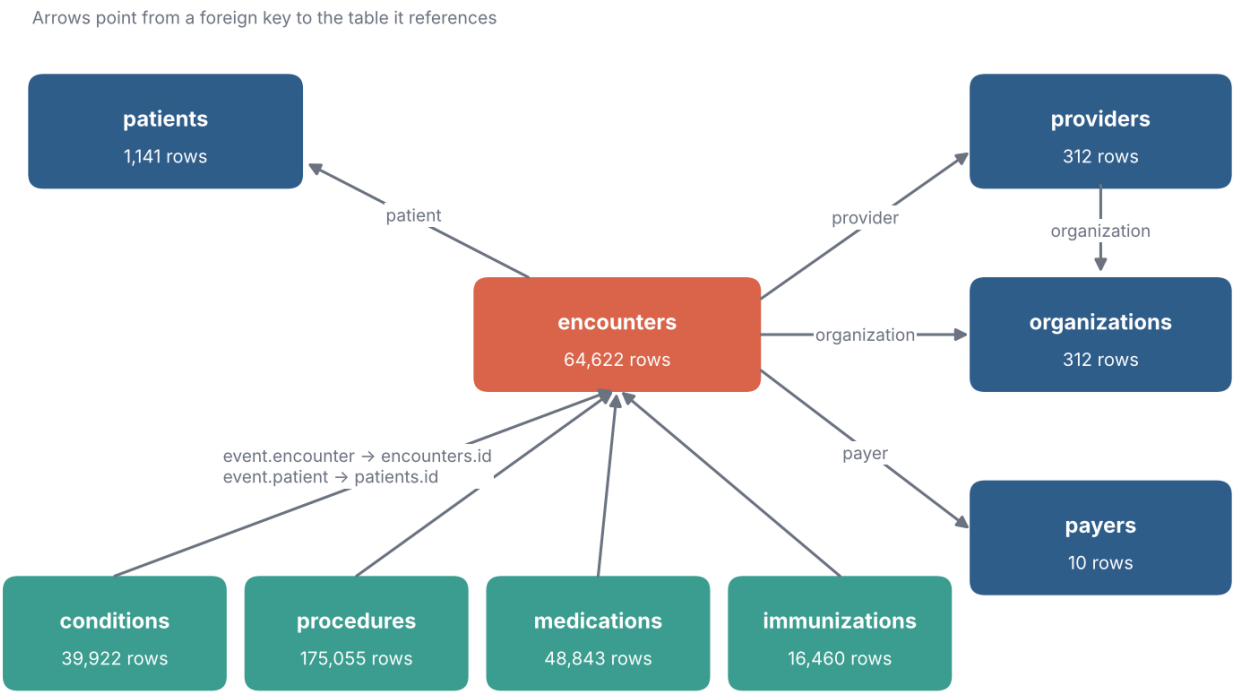


Figure 1. Logical model. Encounters sit at the center; four dimension tables describe them and four event tables hang off them.

Table	Rows	Key	References
patients	1,141	id	—
organizations	312	id	—
providers	312	id	organization → organizations.id
payers	10	id	—
encounters	64,622	id	patient, provider, organization, payer
conditions	39,922	(none: event row)	patient → patients.id; encounter → encounters.id
procedures	175,055	(none: event row)	patient → patients.id; encounter → encounters.id
medications	48,843	(none: event row)	patient, encounter, payer
immunizations	16,460	(none: event row)	patient → patients.id; encounter → encounters.id

Table 3. Tables, keys and references (docs/table_relationships.md)

Event tables have no natural key, so a loader-assigned _row_id identifies every row in every table. That makes each check result traceable to one source row.

4. SQL reporting views

The reporting layer is 8 views in sql/02_reporting_views.sql. Every aggregate view is built on vw_encounter_detail, so all of them share the same joins, the same filters and the same encounter count.

View	Grain	Supports
vw_encounter_detail	Encounter	Base layer: encounter + patient age/gender, provider specialty, organization, payer, duration, reporting-window flag, match flags
vw_encounter_volume_monthly	Month × class	Volume trends
vw_encounter_class_summary	Encounter class	Class mix, patients per class, median duration, average claim cost, payer coverage
vw_visit_type_summary	Class × visit code	Visit types ranked within each class (57 SNOMED codes)
vw_encounter_by_age_gender	Age group × gender × class	Demographic mix
vw_encounter_by_payer	Payer	Volume and share of claim cost covered
vw_patient_utilization	Patient × year	Encounters per patient, acute and inpatient visits
vw_organization_volume	Organization	Site volume and concentration

Table 4. Reporting views

Why LEFT JOINs in the base view

An INNER JOIN would silently drop any encounter whose patient, provider, organization or payer is missing. The base view uses LEFT JOINs and exposes has_patient, has_provider, has_organization and has_payer flags, so unmatched encounters stay visible and are labelled '(unmatched)' in the roll-ups. The trade-off is that LEFT JOINs inflate counts when a dimension key is duplicated, which is why the reconciliation in section 6 checks counts before and after every join.

Example: class summary

```
SELECT encounterclass, COUNT(*) AS encounters,
  ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_encounters,
  COUNT(DISTINCT patient_id) AS unique_patients,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY duration_minutes) AS median_duration_min
FROM vw_encounter_detail GROUP BY encounterclass;
```

5. Encounter volume and visit types

Figures in this section use the 10-year reporting window (Oct 2016 – Sep 2026): 44,424 encounters for 1,046 patients. Synthea also exports older encounters tied to long-running conditions; those are kept in the data but excluded from trend views.

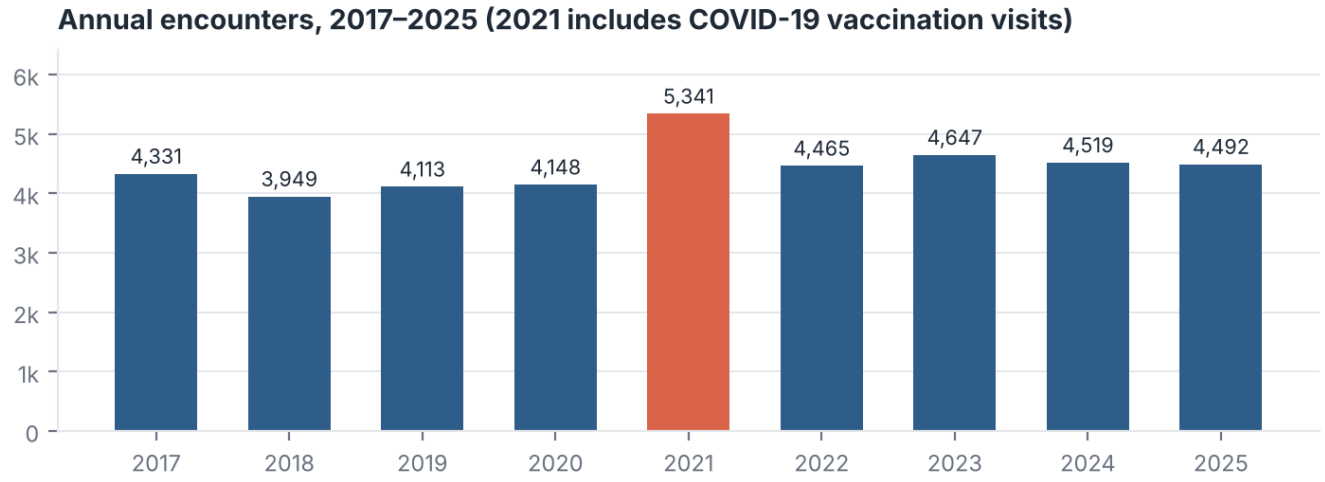


Figure 2. Annual encounters, complete years only

Volume is steady at about 3,949–4,647 encounters a year, or about 4.9 per active patient. 2021 is the exception: Synthea models COVID-19 vaccination, and vaccine-administration encounters jump from 8 in 2020 to 1,180 in 2021 before falling back to 36 in 2022.

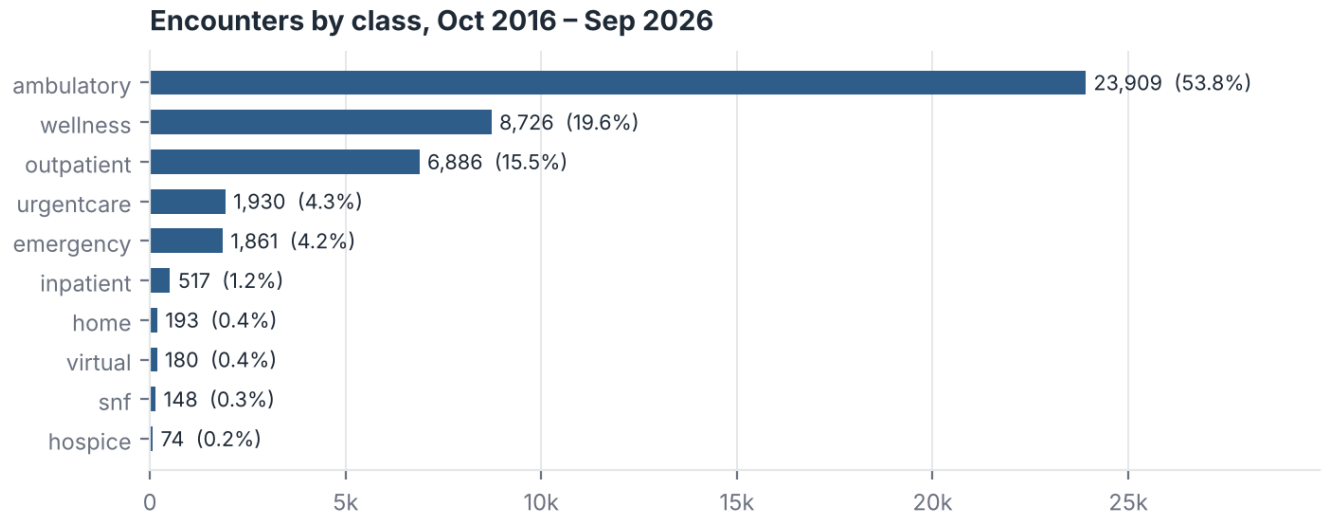


Figure 3. Encounter class mix

Class	Encounters	Share	Patients	Median duration	Avg claim cost	Payer covered
ambulatory	23,909	53.8%	1,025	48 min	$4,282	79%
wellness	8,726	19.6%	1,046	36 min	$1,103	58%
outpatient	6,886	15.5%	944	15 min	$1,548	70%
urgentcare	1,930	4.3%	323	37 min	$965	75%
emergency	1,861	4.2%	686	60 min	$2,901	76%
inpatient	517	1.2%	216	96.0 h	$19,967	91%
home	193	0.4%	11	15 min	$665	87%
virtual	180	0.4%	88	15 min	$616	77%
snf	148	0.3%	135	384.2 h	$17,297	75%
hospice	74	0.2%	68	438.6 h	$12,224	86%

Table 5. Encounter class summary (reporting window)

- **Volume and cost run in opposite directions.** Ambulatory, wellness and outpatient visits make up 89% of encounters. Inpatient, skilled nursing and hospice together are under 2%, but each of those stays costs $12,000–$20,000 on average.
- **Coverage varies by class.** Payers cover 58% of wellness claim cost but 91% of inpatient claim cost.

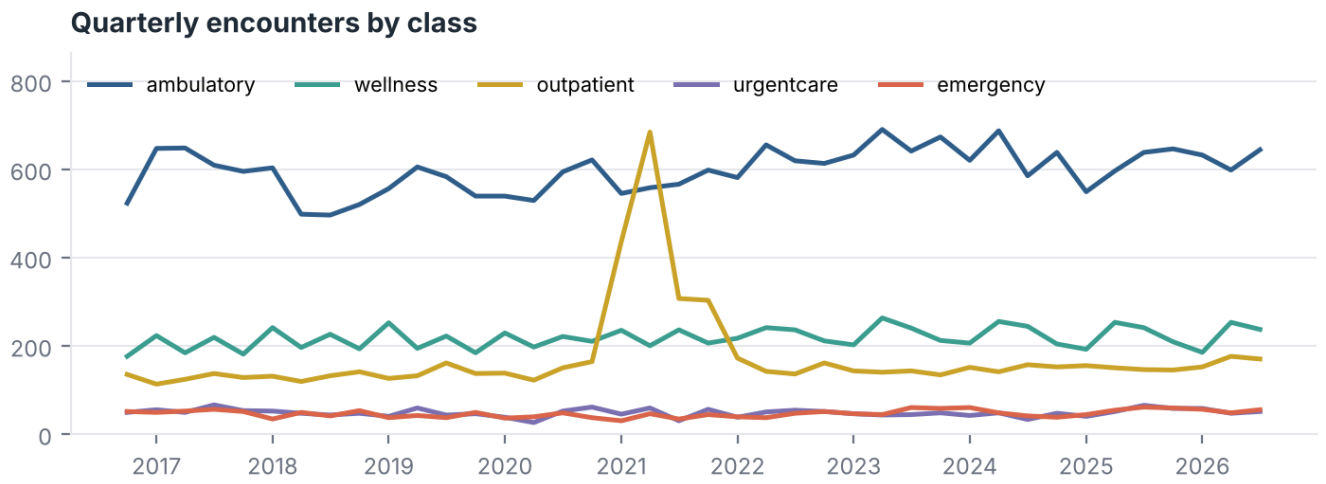


Figure 4. Quarterly encounters for the five largest classes

Class	Visit type	Encounters	Patients
ambulatory	Encounter for problem	7,061	432
wellness	General examination of patient	5,495	828
ambulatory	Encounter for check up	5,448	991
wellness	Well child visit	3,183	356
ambulatory	Prenatal visit	2,884	226
ambulatory	Encounter for symptom	2,749	930
outpatient	Encounter for check up	2,212	506
outpatient	Patient encounter procedure	1,931	239
urgentcare	Urgent care clinic	1,919	315
ambulatory	Outpatient procedure	1,576	35
emergency	Emergency room admission	1,360	551
outpatient	Administration of vaccine to produce active immunity	1,273	689

Table 6. Top 12 of 57 visit types by volume

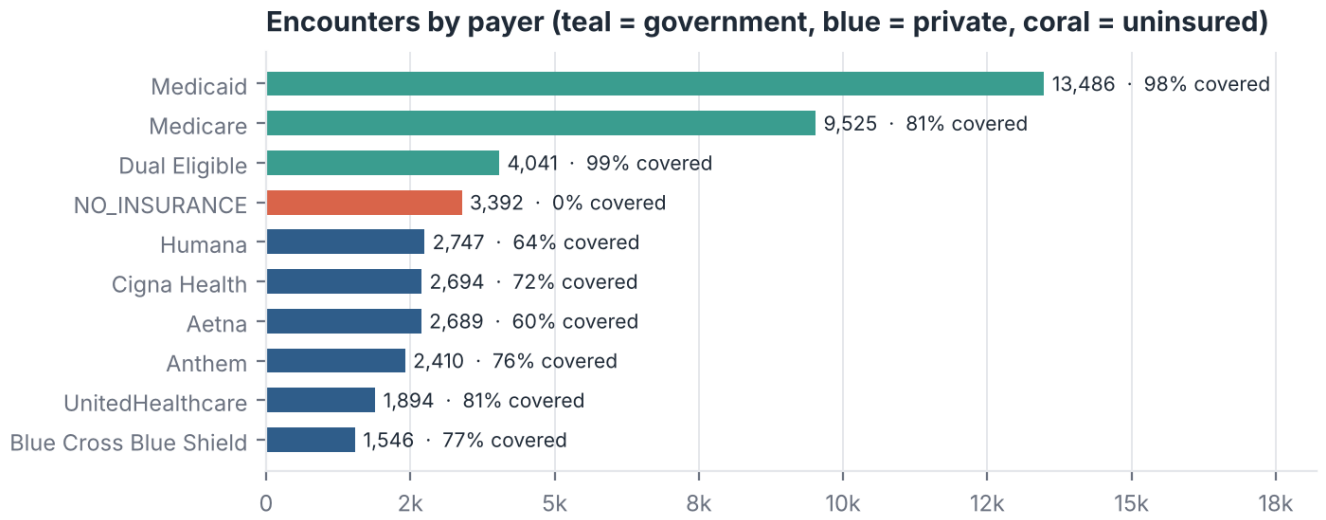


Figure 5. Encounters by payer and share of claim cost covered

Patients and utilization

- **Women account for 67% of encounters.** The gap is widest at ages 18–39, where prenatal care (the fifth-largest visit type) adds thousands of visits.
- **Utilization is skewed.** In 9% of patient-years a patient has 11 or more encounters, mostly driven by repeat care: chronic kidney disease stage 4 (19%), pregnancy (13%) and allergy (9%) are the most common reasons recorded on those encounters.
- **Volume is concentrated.** 15 of 236 organizations account for half of all encounters.

Encounters per patient-year	Patient-years	Share
1	1,427	17.6%
2-3	3,223	39.9%
4-6	1,948	24.1%
7-10	789	9.8%
11+	699	8.6%

Table 7. Utilization distribution, 2017–2025

6. Join reconciliation

Encounter counts are checked before and after each join (sql/03_reconciliation.sql). INNER JOIN steps show how many encounters a join would silently drop; the LEFT JOIN view shows whether duplicate dimension keys inflate the count; the roll-up steps prove each aggregate view adds back to the detail view.

Production data (Synthea as generated)

Step	Join / roll-up	Encounters	vs previous	vs source	Status
1	Source: encounters table	64,622		0	PASS
2	INNER JOIN patients	64,622	0	0	PASS
3	+ INNER JOIN providers	64,622	0	0	PASS
4	+ INNER JOIN organizations	64,622	0	0	PASS
5	+ INNER JOIN payers	64,622	0	0	PASS
6	Reporting view vw_encounter_detail (4 LEFT JOINs)	64,622	0	0	PASS
7	Distinct encounter IDs in vw_encounter_detail	64,622	0	0	PASS
8	Roll-up: SUM(vw_encounter_class_summary.encounters)	64,622	0	0	PASS
9	Roll-up: SUM(vw_encounter_volume_monthly.encounters)	64,622	0	0	PASS
10	Roll-up: SUM(vw_encounter_by_payer.encounters)	64,622	0	0	PASS
11	Roll-up: SUM(vw_patient_utilization.encounters)	64,622	0	0	PASS

Table 8. Production reconciliation: every step returns 64,622 encounters

Test data (with injected errors)

Step	Join / roll-up	Encounters	vs previous	vs source	Status
1	Source: encounters table	64,634		0	PASS
2	INNER JOIN patients	64,814	+180	+180	FAIL
3	+ INNER JOIN providers	67,106	+2,292	+2,472	FAIL
4	+ INNER JOIN organizations	67,096	-10	+2,462	FAIL
5	+ INNER JOIN payers	67,087	-9	+2,453	FAIL
6	Reporting view vw_encounter_detail (4 LEFT JOINs)	67,134	+47	+2,500	FAIL
7	Distinct encounter IDs in vw_encounter_detail	64,622	-2,512	-12	FAIL
8	Roll-up: SUM(vw_encounter_class_summary.encounters)	67,134	+2,512	+2,500	FAIL
9	Roll-up: SUM(vw_encounter_volume_monthly.encounters)	67,134	0	+2,500	FAIL
10	Roll-up: SUM(vw_encounter_by_payer.encounters)	67,134	0	+2,500	FAIL
11	Roll-up: SUM(vw_patient_utilization.encounters)	67,134	0	+2,500	FAIL

Table 9. Test reconciliation

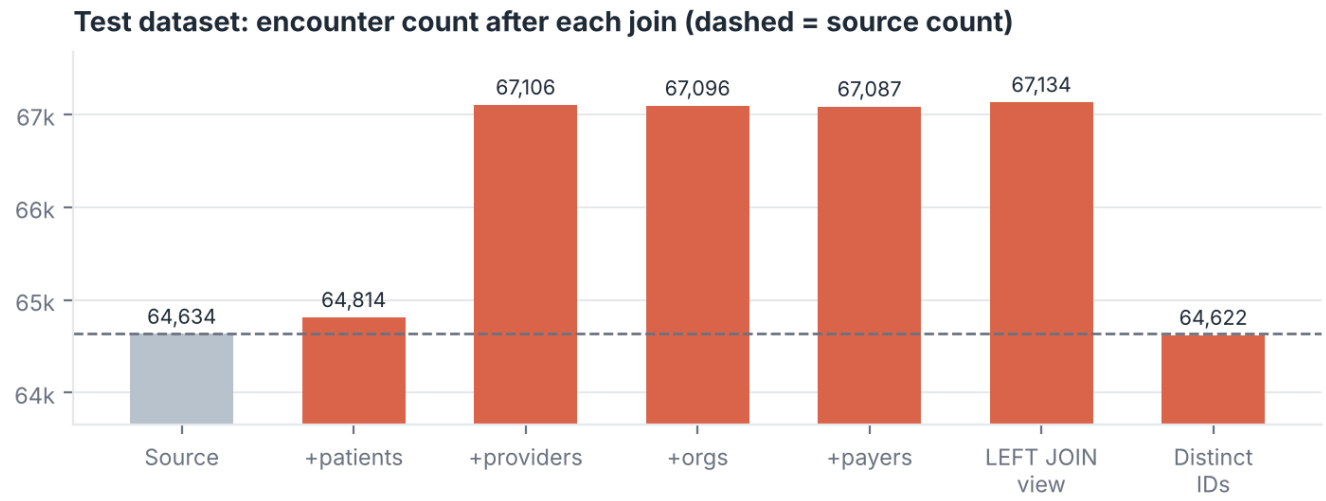


Figure 6. Test dataset: encounter count after each join

Reading the test reconciliation

- **Joining to patients adds 180 rows.** Four patient IDs are duplicated, so every encounter for those patients is counted twice.
- **Joining to providers adds 2,292 more.** Two duplicated provider IDs belong to high-volume providers, so a single duplicate row can double thousands of encounters.
- **Each later INNER JOIN drops rows** where the key is null or orphaned, partly offsetting the inflation, so a row total alone could look close to correct while being wrong in both directions.
- **The LEFT JOIN reporting view shows 67,134 rows for 64,634 source encounters** (+2,500), and only 64,622 distinct encounter IDs because 12 rows reuse an existing ID.

Child linkage

Table	Prod rows	Prod linked	Test rows	Test unlinked	Test linked
conditions	39,922	100.00%	39,922	4	99.99%
procedures	175,055	100.00%	175,059	5	100.00%
medications	48,843	100.00%	48,843	4	99.99%
immunizations	16,460	100.00%	16,462	3	99.98%

Table 10. Share of clinical event rows that join back to an encounter

Takeaway: run the reconciliation before every refresh. A PASS on every step is the gate for publishing the reporting views.

7. Validation checks

The 40 checks in sql/04_dq_checks.sql write every failing row to dq_exceptions (check, table, _row_id, key, detail) and summarize to dq_results. In the production data 36 checks pass and 4 flag genuine source exceptions (section 8).

ID	Category	Table	Rule	Severity	Prod rows	Status
MK01	Missing key	patients	patients.id is null	Critical	0	PASS
MK02	Missing key	encounters	encounters.id is null	Critical	0	PASS
MK03	Missing key	encounters	encounters.patient is null	Critical	0	PASS
MK04	Missing key	encounters	encounters.provider is null	High	0	PASS
MK05	Missing key	encounters	encounters.organization is null	High	0	PASS
MK06	Missing key	encounters	encounters.payer is null	High	0	PASS
MK07	Missing key	providers	providers.organization is null	Medium	0	PASS
MK08	Missing key	conditions	conditions.patient or .encounter is null	High	0	PASS
MK09	Missing key	procedures	procedures.patient or .encounter is null	High	0	PASS
MK10	Missing key	medications	medications.patient or .encounter is null	High	0	PASS
MK11	Missing key	immunizations	immunizations.patient or .encounter is null	High	0	PASS
OR01	Orphan	encounters	encounters.patient not found in patients	Critical	0	PASS
OR02	Orphan	encounters	encounters.provider not found in providers	High	0	PASS
OR03	Orphan	encounters	encounters.organization not found in organizations	High	0	PASS
OR04	Orphan	encounters	encounters.payer not found in payers	High	0	PASS
OR05	Orphan	providers	providers.organization not found in organizations	Medium	0	PASS
OR06	Orphan	conditions	conditions.encounter not found in encounters	High	0	PASS
OR07	Orphan	procedures	procedures.encounter not found in encounters	High	0	PASS
OR08	Orphan	medications	medications.encounter not found in encounters	High	0	PASS
OR09	Orphan	immunizations	immunizations.encounter not found in encounters	High	0	PASS
OR10	Orphan	clinical events	event patient differs from its encounter's patient	High	0	PASS
DU01	Duplicate ID	patients	patients.id appears more than once	Critical	0	PASS
DU02	Duplicate ID	encounters	encounters.id appears more than once	Critical	0	PASS
DU03	Duplicate ID	providers	providers.id appears more than once	High	0	PASS
DU04	Duplicate ID	organizations	organizations.id appears more than once	High	0	PASS
DU05	Duplicate ID	payers	payers.id appears more than once	High	0	PASS
DU06	Duplicate ID	encounters	exact duplicate encounter row (all fields equal)	High	0	PASS
DU07	Duplicate ID	procedures	same patient + encounter + code + start recorded twice	Medium	0	PASS
DU08	Duplicate ID	immunizations	same patient + vaccine code + date recorded twice	Medium	0	PASS
DT01	Date	encounters	encounter stop before start	High	0	PASS
DT02	Date	encounters	encounter start before patient birthdate	Critical	0	PASS
DT03	Date	encounters	encounter start more than 1 day after patient deathdate (excl. Death Certification)	Critical	0	PASS
DT04	Date	encounters	encounter start after data reference date (2026-09-30)	High	131	FAIL
DT05	Date	patients	deathdate before birthdate	Critical	0	PASS
DT06	Date	patients	birthdate after data reference date	Critical	0	PASS
DT07	Date	procedures	procedure starts before its encounter or >24h after encounter stop	Medium	56	FAIL
DT08	Date	procedures	procedure stop before start	Medium	0	PASS

ID	Category	Table	Rule	Severity	Prod rows	Status
DT09	Date	conditions	condition stop before start	Medium	0	PASS
DT10	Date	medications	medication stop before start	Medium	76	FAIL
DT11	Date	immunizations	immunization date differs from its encounter date	Medium	42	FAIL

Table 11. Check catalog with production results

8. Documented exceptions

The untouched Synthea output fails 4 checks on 305 rows. These are real quirks of the source, not injected errors. They are documented, left in place, and subtracted from the test results so they do not count as detections. Full list: docs/exceptions_log.md and reports/output/prod_dq_exceptions.csv.

Check	Rows	What it is	Handling
DT04	131	Encounters dated 1–16 days after the reference date (Synthea schedules a few visits past it)	Kept in source; excluded from the reporting window
DT07	56	Procedures more than 24 h after their encounter closed, mostly digital rectal exams	Kept and flagged; reported to the data owner
DT10	76	Medications whose stop date is 4–6 days before the start date	Kept and flagged; exclude from duration metrics
DT11	42	Vaccine doses (41 of 42 COVID-19) dated 1–35 days after the encounter they are attached to	Kept and flagged; use the immunization date for dose timing

Table 12. Source exceptions found in the production data

Rule adjustments

Two rules flagged expected behaviour in their first version. Each was narrowed with a written rationale rather than deleted, so the check still catches the real error.

Check	Original rule	Rows before	Adjusted rule	Rationale
DT03	Encounter > 1 day after death	118	Excludes encounter code 308646001 (Death Certification)	Death certification is recorded after death by design (2–14 days)
DT07	Procedure outside encounter start–stop	36,728	Procedure before encounter start or > 24 h after stop	Synthea sequences procedures after a short encounter closes; median gap under an hour

Table 13. Check rules adjusted after reviewing production results

9. Testing the checks against injected errors

To show the checks work, python/04_build_test_dataset.py builds a separate copy of the 9 tables and introduces 150 errors, logging each one (table, _row_id, column, old value, new value, expected check) in data/test/injected_error_log.csv. The production copy is never touched, and rows that already fail a check in production are excluded, so every detection can be attributed to an injected error.

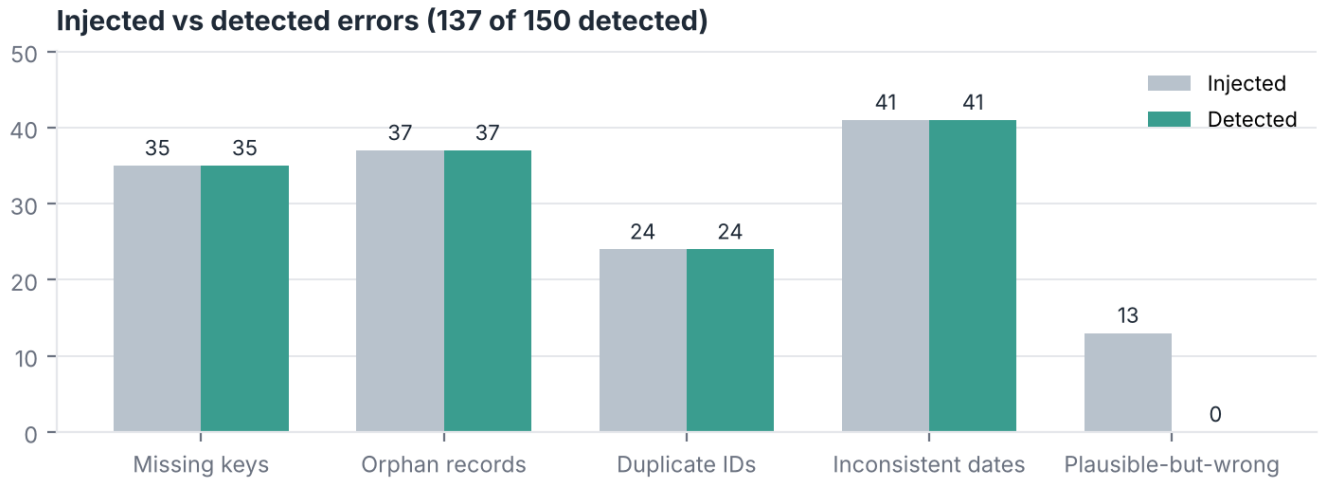


Figure 7. Injected and detected errors by group

137 / 150	137 / 137	137	0
errors detected	structural errors detected	caught by the expected check	untraced false positives

Group	Error type	Expected check	Injected	Detected
Missing keys	null conditions.encounter	MK08	4	4
Missing keys	null encounters.organization	MK05	5	5
Missing keys	null encounters.patient	MK03	8	8
Missing keys	null encounters.payer	MK06	5	5
Missing keys	null encounters.provider	MK04	6	6
Missing keys	null immunizations.encounter	MK11	3	3
Missing keys	null procedures.patient	MK09	4	4
Orphan records	condition linked to wrong patient	OR10	4	4
Orphan records	orphan encounters.organization	OR03	5	5
Orphan records	orphan encounters.patient	OR01	8	8
Orphan records	orphan encounters.payer	OR04	3	3
Orphan records	orphan encounters.provider	OR02	5	5
Orphan records	orphan medications.encounter	OR08	4	4
Orphan records	orphan procedures.encounter	OR07	5	5
Orphan records	orphan providers.organization	OR05	3	3
Duplicate IDs	encounter id reused for a different visit	DU02	4	4
Duplicate IDs	exact duplicate encounter row	DU02/DU06	8	8
Duplicate IDs	immunization recorded twice	DU08	2	2
Duplicate IDs	patient id reused with conflicting demographics	DU01	4	4
Duplicate IDs	procedure recorded twice	DU07	4	4

Group	Error type	Expected check	Injected	Detected
Duplicate IDs	provider id duplicated	DU03	2	2
Inconsistent dates	condition stop before start	DT09	4	4
Inconsistent dates	death date before birth date	DT05	2	2
Inconsistent dates	encounter dated after patient death	DT03	5	5
Inconsistent dates	encounter dated before patient birth	DT02	6	6
Inconsistent dates	encounter dated in the future (year typo)	DT04	5	5
Inconsistent dates	encounter stop before start (swapped)	DT01	8	8
Inconsistent dates	immunization date not on encounter date	DT11	4	4
Inconsistent dates	medication stop before start	DT10	2	2
Inconsistent dates	procedure dated before its encounter	DT07	5	5
Plausible-but-wrong	birth date off by one year	none	3	0
Plausible-but-wrong	encounter assigned to a different valid provider	none	4	0
Plausible-but-wrong	encounter billed to a different valid payer	none	3	0
Plausible-but-wrong	encounter date shifted 20 days (still plausible)	none	3	0

Table 14. Detection by error type

Secondary flags

The test run flagged 451 new rows: the 137 injected rows plus 314 related rows (322 flags, because some rows fail more than one check). Every secondary flag was traced back to an injected error; none are false positives.

Trace	Flags
encounter of a patient whose dates were corrupted	209
clinical event attached to an injected encounter	81
original row of an injected duplicate encounter	20
original row of an injected duplicate event	6
original row of an injected duplicate patient	4
original row of an injected duplicate provider	2

Table 15. Secondary flags by cause

Exceptions: the 13 errors the checks missed

The plausible-but-wrong group was built to be invisible to structural checks: every value is a valid key, a valid date and internally consistent. They were all missed, as expected.

Error type	Count	Why structural checks miss it	What would catch it
Encounter assigned to a different valid provider	4	Provider exists; no rule says which provider is correct	Reconcile to source scheduling / billing records
Encounter billed to a different valid payer	3	Payer exists	Compare to the patient's coverage period (payer_transitions table)
Encounter date shifted 20 days	3	Date is inside the patient's lifetime and the data window	Source-to-target row comparison or audit trail
Birth date off by one year	3	Still before the first encounter	Cross-check age-specific care (e.g. well-child visit timing)

Table 16. Missed errors and the control that would catch each

10. Limitations and next steps

- **Synthetic data.** Synthea is internally consistent by construction, so the production run mostly confirms the checks do not over-flag. The injected test is what demonstrates detection.
- **Structural checks only.** The checks prove keys, relationships and dates are consistent. They cannot prove a value is correct (section 9), which needs a comparison against the source system.
- **Fixed thresholds.** The 24-hour procedure tolerance and the 1-day death tolerance suit Synthea. Real EHR data should have them reviewed with the data owner.
- **No format errors tested.** Injected dates are valid timestamps. Malformed strings would fail at load because columns are typed; a staging table of raw text would be needed to test those.

Next steps

- Add a coverage-period check using Synthea's payer_transitions table to catch encounters billed to the wrong payer.
- Schedule the reconciliation and checks after every load, and block publishing when a Critical check fails.
- Track dq_results over time to show trends by check, not only point-in-time counts.

11. Reproducing the analysis

Requirements: Java 17+, PostgreSQL 14+, Python 3.10+ with pandas, psycopg, matplotlib and reportlab.

```
pip install -r requirements.txt
createdb encounters_dq
export PGHOST=localhost PGPORT=5432 PGUSER=postgres PGDATABASE=encounters_dq
bash run_all.sh    # regenerate Synthea data and rebuild everything
bash run_all.sh --skip-synthea  # reuse the committed data/raw files
```

Path	Contents
synthea/generate_synthea.sh	Seeded Synthea run (jar downloaded on first use, not committed)
data/raw/	9 tables as generated (gzip CSV)
data/test/	9 tables with injected errors + injected_error_log.csv (ground truth)
sql/01–04	Schema, reporting views, join reconciliation, validation checks
python/01–07	Prepare, load, check, inject, score, export, report
docs/	Table relationships, exceptions log, check catalog
reports/output/	Every result table behind this report (CSV)

Table 17. Repository contents